django

创建启动

```
mkvirtualenv baizhan_env
pip install django==4.2.3
django-admin startproject edu_project
python manage.py runserver

```

改数据库

- /setting.py

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'edu_db',
        'USER':'root',
        'PASSWORD':'lin1234567',
        'HOST':'localhost',
        'POST':'3306',
    }
}
```

用户创建

下载链接数据库的包

```
pip install pymysql
```

- __ init __.py

```
import pymysql

pymysql.install_as_MySQLdb()
```

```
python .\manage.py startapp user_app
```

写model

- /user/models.py
- https://www.cnblogs.com/fmgao-technology/p/9989261.html#:~:text=2%E3%80%81models.CharField%20%2D%2D%2D%E5%AD%97%E7%AC%A6,%E5%85%81%E8%AE%B8%E7%9A%84%E6%9C%80%E5%A4%A7%E5%AD%97%E7%AC%A6%E6%95%B0%E3%80%82

```
import hashlib


```

```
from django.db import models


class User(models.Model):


  GENDER_CHOICES = (
     (0, '女'),
     (1, '男')
   )
  phone = models.CharField(null=True,max_length=11,verbose_name='手机号')
  _password = models.CharField(null=True,blank=True,max_length=100,verbose_name='真实密码')
  password = models.CharField(null=True,blank=True,max_length=100,verbose_name='密码',db_column=None)
  nickname= models.CharField(null=True,max_length=50,blank=True,verbose_name='昵称')
  gender = models.IntegerField(null=True,blank=True,verbose_name='性别',choices=GENDER_CHOICES,default=1)
  job_title = models.CharField(null=True,max_length=50,blank=True,verbose_name='职称')
  introduction = models.TextField(null=True,blank=True,verbose_name='简介')
  avatar = models.CharField(null=True,blank=True,max_length=50,verbose_name='头像')


  create_at = models.DateTimeField(auto_now_add=True,verbose_name='创建时间')
  update_at = models.DateTimeField(auto_now=True,verbose_name='更新时间')


  class Meta:
    db_table = 't_user'


  @property
  def password(self):
    return self._password


  @password.setter
  def password(self, pwd):
    # 数据密码数据加密
    self._password = hashlib.md5(pwd.encode()).hexdigest()


  def check_password(self, raw_password):
    encrypted = hashlib.md5(raw_password.encode()).hexdigest()
    return encrypted == self._password

```

迁移数据库

```
python .\manage.py makemigrations
python .\manage.py migrate
```

- 遇到数据库版本过低的报错可以

D:\python_venv\django_demo\Lib\site-packages\django\db\backends\base\base.py

```
    def init_connection_state(self):
        """Initialize the database connection settings."""
        global RAN_DB_VERSION_CHECK
        if self.alias not in RAN_DB_VERSION_CHECK:
           #  self.check_database_version_supported()   ------- 把这一行注释
            RAN_DB_VERSION_CHECK.add(self.alias)

```

编写接口

- 一般在**views.py**中

```
pip install djangorestframework==3.14.0
```

objects函数

- 你可以执行查询、创建、更新和删除等操作

```
from myapp.models import User

# 获取所有用户
users = User.objects.all()

# 获取所有性别为男的用户
male_users = User.objects.filter(gender=1)

# 获取手机号为特定值的用户
user = User.objects.get(phone='12345678901')
# 按创建时间降序排列
users = User.objects.order_by('-create_at')

# 按昵称升序排列
users = User.objects.order_by('nickname')
# 获取前10个用户
top_10_users = User.objects.all()[:10]
```

- 创建逻辑

```
from rest_framework.views import APIView
from user_app.models import User
from rest_framework.response import Response
from rest_framework import status
# Create your views here.
```

```
class LoginView(APIView):
    def post(self,request):
        """
        登录
        """
        # 获取参数
        phone = request.data.get('phone')
        password = request.data.get('password')
        # 查找数据
        try:
            user = User.objects.get(phone=phone)
        except User.DoesNotExist:
            return Response({'code':status.HTTP_404_NOT_FOUND,'msg':'用户不存在'})
        # 校验
        if user.check_password(password):
            return Response({
                'code':status.HTTP_200_OK,
                'msg':'登陆成功',
                'nickname':user.nickname,
                'user_id':user.id
            })
        return Response({'code':status.HTTP_400_BAD_REQUEST,'msg':'密码错误'})

        # return
```

创建路由

- 当前文件夹下创建url.py

```
from django.urls import path

from . import views

urlpatterns = {
    path('login/',views.LoginView.as_view()),
}
```

- 主应用下的urls.py

```
from django.urls import path,include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/',include('user_app.url')),
]
```

注册

```

```

用户信息

报错

```
AssertionError: Expected view UserDetail to be called with a URL keyword argument named "pk". fix your URl
url地址上的参数没有定义成PK
序列化时没有在模型化定义
```

- 把路由改了

```
path('user/<int:pk>/',views.UserDetail.as_view()),
```

serializers序列化

- 处理数据和复杂数据类型的转换工具，类似jango的表单类，将复杂的数据类型转化成本地的python类型方便呈现为json
- 帮我们验证数据，序列化数据，反序列化数据

```
from rest_framework import serializers
from .models import User


# 创建注册序列化器
class RegisterSerializer(serializers.ModelSerializer):
    phone = serializers.CharField(max_length=11, min_length=11, required=True)
    password = serializers.CharField(max_length=32, min_length=3, required=True)
    nickname = serializers.CharField(required=False)
    gender = serializers.IntegerField(required=False)
    job_title = serializers.CharField(required=False)
    introduction = serializers.CharField(required=False)
    avatar = serializers.CharField(required=False)

    # 验证手机号是否已经注册
    def validate_phone(self, value):
        try:
            User.objects.get(phone=value)
            raise serializers.ValidationError('手机号已经注册')
        except User.DoesNotExist:
            return value
        return value

```

```
    class Meta:# 元数据
        model = User
        fields = '__all__' # 选择全部的数据去
```

rest_framework

```
# RetrieveAPIView找单一的某一个数据区别
```

ListAPIView和APIView的区别

`generics.ListAPIView` 是 DRF 提供的 **高度封装的通用视图**，它已经内置了 `.get()` 方法的逻辑。

当你继承 `ListAPIView` 时：

- 它**自动实现了** `get` **请求**的处理逻辑。
- 它会自动使用你定义的 `queryset` 作为数据源。
- 自动使用 `serializer_class` 进行序列化。
- 自动应用 `pagination_class` 分页。
- 所以你**不需要自己写** `get()` **方法**。

> 🔹 相当于：你只负责"配置"，DRF 帮你"执行"。

属性名	作用
`queryset`	指定要列出的数据集
`serializer_class`	指定用哪个序列化器来返回数据
`pagination_class`	指定分页规则

因为 `APIView` 是 **最基础的视图类**，它不会自动帮你实现任何 HTTP 方法的逻辑。

- 它**不假设你要做什么**（是注册？登录？上传？）。
- 所以你必须**手动实现** `post()` **方法**来处理 POST 请求。
- 它也没有默认的 `queryset` 或 `serializer_class` 行为，除非你自己在方法中使用。

> 🔹 相当于：你负责"从零写逻辑"，DRF 只提供工具（如 Request、Response、认证等）。

url调度器

```
from django.urls import path


urlpatterns = [
  path('user/',user),
  path('user/info/',user_info),
  path('user/<id>/',user_id),       # 当成参数传入到变量里
  path('user/<id>/<year>/',user_id_year),
  path('user/<int:id>/',user_int),
]

```

路径转换器

- str：匹配任何非空字符串，不包括路径分隔符'/'。如果转换器不包含在表达式中，这是默认值。
- int：匹配零或任何正整数。返回一个int。
- slug：匹配由ASCII字母或数字组成的字符串，以及横线和下划线字符。例如： building-your-1st-django_site可以匹配，django_@site是不可以匹配的。
- uuid：匹配格式化的UUID。为防止多个URL映射到同一页面，必须包含破折号，并且字母必须是小写。例如，075194d3-6885-417e-a8a8-6c931e272f00。返回一个 UUID实例。

```
path('articles/<uuid:uuid>/',views.article_uuid),
```

- path：匹配任何非空字符串，包括路径分隔符 '/'，可以匹配完整的URL路径，而不仅仅是URL路径的一部分str，使用时要谨慎，因为可能造成后续的所有url匹配都失效。

自定义url

- 一个regex类属性，作为一个re匹配字符串
- to_python(self, value)方法，它处理匹配的字符串转换成要传递到视图函数的类型
- to_url(self, value)方法，用于处理将Python类型转换为URL中使用的字符串

新建一个converters.py文件，在文件中定义一个FourDigitYearConverter类：

```
class FourDigitYearConverter(object):
  regex = '[0-9]{4}'


  def to_python(self, value):
    return int(value)


  def to_url(self, value):
    return '%04d' % value

```

使用register_converter()方法在URLs中注册自定义转换器类：

```
from django.urls import register_converter, path


from . import converters, views


register_converter(converters.FourDigitYearConverter, 'yyyy')


urlpatterns = [
  path('articles/2030/', views.special_case_2030),
  path('articles/<yyyy:year>/', views.year_archive)
]

```

URL调度器-错误处理 自定义错误界面

urls 中配置

```
# polls是子应用
handler404 = "polls.views.page_not_found"

```

再polls应用中views中添加函数

```
def page_not_found(request, exception):
  return HttpResponse('自定义的404错误页面')
```

自定义页面

```
def page_not_found(request, exception):
    print("enter")
    return render(request, 'user/a404.html', status=404)
```

设置首页

```
urlpatterns = [
    path("", index, name="index"),  # 添加首页路由
```

设置首页

```
在setting中加
# 添加静态文件根目录
STATICFILES_DIRS = [
    BASE_DIR / "static",
]


在views中设置首页视图
# 添加首页视图
def index(request):
    return render(request, 'home/index.html')

更新url
urlpatterns = [
    path("", index, name="index"),  # 添加首页路由
```

引入其他路由

include(str)

```
from django.urls import include, path


urlpatterns = [
  path('community/', include('aggregator.urls')),
  path('contact/', include('contact.urls')),
]

```

include(list/tuple)

```
from django.urls import include, path


from apps.main import views as main_views
```

```
from credit import views as credit_views


extra_patterns = [
  path('reports/', credit_views.report),
  path('reports/<int:id>/', credit_views.report),
  path('charge/', credit_views.charge),
]


urlpatterns = [
  path('', main_views.homepage),
  path('help/', include('apps.help.urls')),
  path('credit/', include(extra_patterns)),
]

```

优化

```
from django.urls import path
from . import views


urlpatterns = [
  path('<page_slug>-<page_id>/history/', views.history),
  path('<page_slug>-<page_id>/edit/', views.edit),
  path('<page_slug>-<page_id>/discuss/', views.discuss),
  path('<page_slug>-<page_id>/permissions/', views.permissions),
]


等价于
from django.urls import include, path
from . import views


urlpatterns = [
  path('<page_slug>-<page_id>/', include([
    path('history/', views.history),
    path('edit/', views.edit),
    path('discuss/', views.discuss),
    path('permissions/', views.permissions),
   ])),
]

```

models

ORM框架

```
from django.db import models


class Person(models.Model):
  first_name = models.CharField(max_length=30)
  last_name = model
  s.CharField(max_length=30)

===
CREATE TABLE myapp_person (
  "id" bigint NOT NULL PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  "first_name" varchar(30) NOT NULL,
  "last_name" varchar(30) NOT NULL
);

```

django会自己创建id的主键

1. 模型类必须继承models.Model
2. 每个属性对应数据库表中的一个字段
3. 表名自动使用 应用_类名 的小写（如：polls_question），可以覆盖重写
4. 如果模型类中没有指定 primary_key，那么会自动创建一个 id 字段，自增，主键

数据库设置

1. 创建项目model_study,及子应用model_app

```
#创建项目
$ django-admin startproject model_study
#进入项目目录创建子应用
$ python manage.py startapp model_app
```

2. 配置应用，将模型对应的应用程序添加到项目的settings中：

```
INSTALLED_APPS = [
  'model_app'
]
```

3. 在settings.py中配置正确的数据库连接：

```
# mysql
DATABASES = {
  'default': {
    'ENGINE': 'django.db.backends.mysql',
    'NAME': 'model_study',
    'USER': 'root',
    'PASSWORD': 'root',
    'HOST': '127.0.0.1',
    'PORT': 3306,
   }
}
```

```
pip install mysqlclient==2.1.1

pip install pymysql
```

逆向models

```
python manage.py inspectdb > model_app/models.py


```

字段

常见的字段

字段命名限制

- 字母，数字，下划线，首字母不能是数字
- 字段名称不能是Python保留字
- 由于Django查询查找语法的工作方式，字段名称不能在一行中包含多个下划线，譬如“abc__123”就是不允许的，一个下划线是可以的，如：'first_name'

官方文档：https://docs.djangoproject.com/zh-hans/4.1/ref/models/fields/#field-types

字段名	作用
AutoField	自增一个IntegerField，根据可用的 ID 自动递增
BooleanField	该字段的默认表单部件是checkbox,默认值是 None
CharField	一个字符串字段
DateField	一个日期，在 Python 中用一个 `datetime.date` 实例表示
DateTimeField	一个日期和时间，在 Python 中用一个 `datetime.datetime` 实例表示
FloatField	在 Python 中用一个 `float` 实例表示的浮点数
SmallIntegerField	就是一个 IntegerField， `-32768` 到 `32767` 的值
IntegerField	一个整数。从 `-2147483648` 到 `2147483647` 的值

字段名	作用
TextField	一个大的文本字段。该字段的默认表单部件是一个Textarea

常见的属性

- max_length：字段最大长度，用于字符串等，字符串类型CharField必须设置该值
- null：如果True，Django将在数据库中存储NULL空值。默认是False
- blank：如果True，该字段被允许为空白("")。默认是False。
- choices：

  示例：YEAR_IN_SCHOOL_CHOICES = (('FR', 'Freshman'),('SO', 'Sophomore'),('JR', 'Junior'),('SR', 'Senior'),('GR', 'Graduate')) ,

  中文示例：SEX_CHOICES=((1, '男'),(2, '女'))

  元组中的第一个元素是将存储在数据库中的值，第二个元素是将在页面中显示的值，最常见用于下拉选择框select
- default：字段的默认值
- help_text：用于显示额外的“帮助”文本
- primary_key：如果True，这个字段是模型的主键，默认是False
- unique：如果True，该字段在整个表格中必须是唯一的
- verbose_name：详细字段名，不指定则是属性名的小写，并且用 空格 替换 '_'

Django模型-1对1关系映射

关系型数据库的强大之处在于各表之间的关联关系。 Django 提供了定义三种最常见的数据库关联关系的方法：多对一，多对多，一对一

一对一关联使用 OneToOneField来定义一对一关系

就像使用其他类型的 Field 一样：在**任意一边模型**属性中包含它

```
from django.db import models


class Place(models.Model):
  name = models.CharField(max_length=50)
  address = models.CharField(max_length=80)
```

```


  def __str__(self):
    return "%s the place" % self.name


class Restaurant(models.Model):
  place = models.OneToOneField(
    Place,
    on_delete=models.CASCADE,
    primary_key=True,
   )
  # BooleanField 在数据库使用 tinyint 类型
  serves_hot = models.BooleanField(default=False)
  serves_clod= models.BooleanField(default=False)


  def __str__(self):
    return "%s the restaurant" % self.place.name


```

Django模型-1对多关系映射

定义一个多对一的关联关系，使用 django.db.models.ForeignKey 类。就和其它 Field字段类型一样，只需要在你模型中添加一个值为该类的属性

ForeignKey类需要**多的一方**添加一个参数，即你想要关联的模型类名

```
from django.db import models


class Place(models.Model):
  name = models.CharField(max_length=50)
  address = models.CharField(max_length=80)


  def __str__(self):
    return "%s the place" % self.name
```

```
class Restaurant(models.Model):
  place = models.OneToOneField(
    Place,
    on_delete=models.CASCADE,
    primary_key=True,
   )
  # BooleanField 在数据库使用 tinyint 类型
  serves_hot_dogs = models.BooleanField(default=False)
  serves_pizza = models.BooleanField(default=False)


  def __str__(self):
    return "%s the restaurant" % self.place.name


class Waiter(models.Model):
  restaurant = models.ForeignKey(Restaurant, on_delete=models.CASCADE)
  name = models.CharField(max_length=50)


  def __str__(self):
    return "%s the waiter at %s" % (self.name, self.restaurant)


```

Django模型-多对多关系映射

定义一个多对多的关联关系，使用 django.db.models.ManyToManyField 类。就和其他Field字段类型一样，只需要在你模型中添加一个值为该类的属性

ManyToManyField类需要**任意模型边**添加一个位置参数，即你想要关联的模型类名

```
class SchoolClass(models.Model):
  name = models.CharField(max_length=20)


class Teacher(models.Model):
  name = models.CharField(max_length=10)
  school_class = models.ManyToManyField(SchoolClass)

```

Django模型-数据的查询介绍

查询方式

- `Model.objects.get( )` 返回一个匹配的对象
- `Model.objects.all( )` 返回一个`QuerySet`,包含所有数据
- `Model.objects.filter( )` 返回一个新的`QuerySet`，包含复合规则的
- `Model.objects.exclude( )` 返回一个新的`QuerySet`，不包含指定规则的 取反

情况	是否连表查询	说明
`Model.objects.get()`	❌ 否	只查本表
`Model.objects.select_related().get()`	✅ 是	主动 JOIN 关联表
访问关联字段（如 `obj.foreign_key.field`）	❌ 惰性加载	第一次用时才查，可能造成 N+1 问题

Django模型-数据的条件查询

参考文档：https://docs.djangoproject.com/zh-hans/4.1/ref/models/querysets/#field-lookups

字段检索，是在**字段名**后加 '__' 双下划线，再加关键字，类似 SQL 语句中的 where 后面的部分， 如： 字段名__关键字

- exact ： 判断是否等于value，一般不使用，而直接使用 '='
- contains：是否包含,大小写敏感，如果需要不敏感的话，使用icontains
- startswith：以value开头,大小写敏感
- endwith：以value结尾,大小写敏感
- in：是否包含在范围内
- isnull：是否为null， 如：filter(name__isnull=Flase)
- gt：大于，如：filter(sage__gt=30) ， 年龄大于30
- gte：大于等于
- lt：小于
- lte：小于等于

```
# 获取ID等于6
Waiter.objects.filter(id__exact=6)
# 获取ID等于6
Waiter.objects.filter(id=6)
# 获取name名字包含"张"
Waiter.objects.filter(name__contains="张")
# 获取name名字包含"吕"
Waiter.objects.filter(name__contains="吕")
# 获取name名字以"袁"开头的
Waiter.objects.filter(name__startswith="袁")
# 获取name名字以"辽"结尾的
Waiter.objects.filter(name__endswith="辽")
# 获取name名字是"关羽和黄忠"的
Waiter.objects.filter(name__in=["关羽","黄忠"])
# 获取name名字为空的
Waiter.objects.filter(name__isnull=True)
# 获取id大于5的
Waiter.objects.filter(id__gt=5)
# 获取id小于5的
Waiter.objects.filter(id__lt=5)
# 获取id小于等于5的
Waiter.objects.filter(id__lte=5)
# 获取在id为1的餐厅工作的
Waiter.objects.filter(restaurant=1)
# 获取在id为1的餐厅工作的
Waiter.objects.filter(restaurant_id=1)
# 获取在name为肯德基的餐厅工作的
Waiter.objects.filter(restaurant__name="肯德基")
# 会报错，没有用俩个下划线！！！
Waiter.objects.filter(restaurant_name="肯德基")

```

执行原生sql

```
from django.db import connection


cursor = connection.cursor()
cursor.execute("UPDATE t_cook SET level = 1 WHERE id = %s", [1])
cursor.execute("SELECT * FROM t_cook WHERE id = %s", [1])
row = cursor.fetchone()

```

Django视图-FBV和CBV

FBV 是基于函数的视图 （function base views）

CBV 是基于类的视图（class base views）

FBV

就是在视图里使用函数处理请求

```
# urlconf 中
urlpatterns = [
    path('fbv/', views.current_datetime),
]


# views 中
from django.http import HttpResponse
import datetime


def current_datetime(request):
  now = datetime.datetime.now()
  html = "<html><body>It is now %s.</body></html>" % now
  return HttpResponse(html)
```

- 视图函数 current_datetime，每个视图函数都将一个HttpRequest 对象作为其第一个参数，该参数通常被命名request
- 视图函数的名称无关紧要，它不必以某种方式命名，以便Django能够识别它，但是函数命名一定要能够清晰的描述它的功能
- 视图函数返回一个HttpResponse响应的对象，每个视图函数负责返回一个HttpResponse对象（有例外，但我们在稍后讨论）

```
# urlconf 中
urlpatterns = [
  # 一定要使用 as_view() ，记住 小括号
    path('cbv/', views.MyView.as_view()),
]


# views中
from django.http import HttpResponse
from django.views import View

class MyView(View):


  def get(self, request):
    return HttpResponse('get OK')


  def post(self, request):
    return HttpResponse('post OK')

```

- CBV提供了一个as_view()静态方法（也就是类方法），调用这个方法，会创建一个类的实例，然后通过实例调用dispatch()方法，dispatch()方法会根据request的method的不同调用相应的方法来处理request（如get()，post()等）
- 提高了代码的复用性，可以使用面向对象的技术，比如Mixin（多继承）
- 可以用不同的函数针对不同的HTTP方法处理，而不是通过很多 if 判断，可以提高代码可读性

@login_required

必须登录才能访问装饰的视图函数，

用户未登录，则重定向到settings.LOGIN_URL，除非指定了login_url参数，例如：@login_required(login_url='/polls/login/')

```
@login_required
def my_view(request):
  # I can assume now that only GET or POST requests make it this far
  # ...
  pass
```

Django视图-请求对象HttpRequest

每一个用户请求在到达视图函数的同时，Django 会创建一个HttpRequest对象并把这个对象当做第一个参数传给要调用的views方法。HttpRequest对象包含了请求的元数据,比如(本次请求所涉及的用户浏览器端数据、服务器端数据等)，在views里可以通过request对象来调用相应的属性

所有视图函数的第一个参数都是HttpRequest实例

官网：https://docs.djangoproject.com/zh-hans/4.1/ref/request-response/#django.http.HttpReques

- HttpRequest.scheme：

  表示请求使用的协议（http或https）

- HttpRequest.body：

  原始HTTP请求主体，类型是字节串。处理数据一些非html表单的数据类型很有用，譬如：二进制图像，XML等；

  - 取form表单数据，请使用 HttpRequest.POST
  - 取url中的参数，用HttpRequest.GET

- HttpRequest.path：

  表示请求页面的完整路径的字符串，不包括scheme和域名。

  例： "/music/bands/the_beatles/"

- HttpRequest.method：

  表示请求中使用的HTTP方法的字符串，是大写的。例如：

```
if request.method == 'GET':
  do_something()
elif request.method == 'POST':
  do_something_else()
```

- HttpRequest.encoding：

  表示当前编码的字符串，用于解码表单提交数据（或者None，表示使用该DEFAULT_CHARSET设置）。

  可以设置此属性来更改访问表单数据时使用的编码，修改后，后续的属性访问（例如读取GET或POST）将使用新encoding值。

- HttpRequest.content_type：

  表示请求的MIME类型的字符串，从CONTENT_TYPE解析 。

- HttpRequest.content_params：

  包含在CONTENT_TYPE 标题中的键/值参数字典。

- HttpRequest.GET：

  包含所有给定的HTTP GET参数的类似字典的对象。请参阅QueryDict下面的文档。

- HttpRequest.POST：

  包含所有给定HTTP POST参数的类似字典的对象，前提是请求包含表单数据。请参阅QueryDict文档。POST不包含文件信息，文件信息请见FILES。

- HttpRequest.COOKIES：

  包含所有Cookie的字典，键和值是字符串。

- HttpRequest.FILES：

  包含所有上传文件的类似字典的对象

- HttpRequest.META：

  包含所有可用HTTP meta的字典

中间件设置的属性：

Django的contrib应用程序中包含的一些中间件在请求中设置了属性。如果在请求中看不到该属性，请确保使用了相应的中间件类MIDDLEWARE

- HttpRequest.session：

  来自SessionMiddleware：代表当前会话的可读写字典对象。

- HttpRequest.site：

  来自CurrentSiteMiddleware： 代表当前网站的实例Site或 RequestSite返回get_current_site()

- HttpRequest.user：

  来自AuthenticationMiddleware：AUTH_USER_MODEL代表当前登录用户的实例

属性	说明
`request.GET`	URL 查询参数（如 `?name=alice&age=20`）
`request.POST`	表单数据（非文件字段，如文本输入）
`request.FILES`	上传的文件（如图片、文档）

属性	说明
`request.body`	原始请求体内容（字节格式），比如 JSON 字符串或 XML
`request.META`	HTTP 请求头和其他元数据（如 `HTTP_USER_AGENT`, `REMOTE_ADDR`）
`request.method`	请求方法（GET、POST、PUT 等）
`request.path`	请求的路径（如 `/upload/`）
`request.user`	当前登录用户（如果已认证）

✅ 举个例子说明

假设你有一个表单：

html

深色版本

```
<form method="post" enctype="multipart/form-data">
  <input type="text" name="title" value="我的图片">
  <input type="file" name="image" />
  <button type="submit">上传</button>
</form>
```

当用户提交时：

- `request.POST['title']` → `"我的图片"`（表单文本）
- `request.FILES['image']` → 一个 `UploadedFile` 对象（上传的图片文件）
- `request.method` → `"POST"`
- `request.body` → 原始的 `multipart/form-data` 数据流（包含文本和文件的混合编码）

模板

作为一个Web框架，Django需要一种方便的方式来动态生成HTML。最常用的方法依赖于模板。模板包含**所需HTML输出的静态部分**以及描述如何插入**动态内容的特殊语法**

```
TEMPLATES = [
   {
    'BACKEND': 'django.template.backends.django.DjangoTemplates',
    "DIRS": [BASE_DIR / 'templates'],
    'APP_DIRS': True,
    'OPTIONS': {
      'context_processors': [
        'django.template.context_processors.debug',
        'django.template.context_processors.request',
        'django.contrib.auth.context_processors.auth',
        'django.contrib.messages.context_processors.messages',
       ],
     },
   },
```

```
]

```

- BACKEND：是实现Django模板后端API的模板引擎类的路径。内置是 django.template.backends.django.DjangoTemplates和 django.template.backends.jinja2.Jinja2（使用这个需要额外安装jinja2库）
- DIRS ：按搜索顺序定义引擎应该查找模板源文件的目录列表
- APP_DIRS：是否去子应用寻找模板
- OPTIONS：包含后端特定的设置

rest_framework

官网：https://www.django-rest-framework.org/

中文文档：https://q1mi.github.io/Django-REST-framework-documentation/

settings配置

首先新建一个django项目

然后如果要启用REST framework，那么需要将其添加到

INSTALLED_APPS 中

```
INSTALLED_APPS = [
  ...
  'rest_framework'
]
```

序列化和反序列化

序列化：把一个对象（model）转换成json

反序列化：一组一组的数据 封装成objects

创建序列化类

在子应用的目录下，新建app_serializers.py 文件，在其中建立一个对应第一步建立的模型的序列化类：

```
from rest_framework import serializers
from rest_app.models import *

class StudentSerializer(serializers.ModelSerializer):
  class Meta:
    model = Student#要继承哪个模型
    fields = ['id', 'name', 'age','sex'] # 相应字段
    # fields = '__all__'
    # 不序列化 exclude = ["id"]
```

序列化

```
# 得到一个模型实例
stu = Student.objects.get(pk=1)  # stu从数据库取一个
# 得到模型序列化类实例
stu_ser = StudentSerializer(stu) # stu是填充的
# 得到 模型的字典数据 {"id": 1, ......}
data_dict = stu_ser.data


# 转成bit类型
from rest_framework.renderers import JSONRenderer
data_json = JSONRenderer().render(data_dict)

```

- 序列化模型的查询集QuerySet
- many 传递了多个对象

```
# 必须指定 many=True 传递了多个对象
stu_ser = StudentSerializer(Student.objects.all(), many=True)
# 得到 字典数据列表 [OrderedDict([('id', 1),......)]
data_dict = stu_ser.data
```

反序列化

```
# 将 json格式的字节串 转换为字典
from rest_framework.parsers import JSONParser
import io
stream = io.BytesIO(b'{"name":"rose", "age":19, "sex":2}')
# 得到字典数据， {'id': 1,......}
data_dict = JSONParser().parse(stream)


# 将字典数据 反序列化
serializer = StudentSerializer(data=data_dict)
# 必须执行这一步验证， 返回True 执行 save等方法
serializer.is_valid()
# 保存到数据库中
serializer.save()
```

```

```

APIView

原来:

```
from .models import Student
from .serializers import StudentSerializer
from django.http import JsonResponse,HttpResponse
from rest_framework.parsers import JSONParser
from django.views.decorators.csrf import csrf_exempt

'''
对学生执行 增删改查API:
   行为        请求方式    请求路径URL
   增加        POST     /students/
   删除        DELETE    /student/<int:id>/
   修改        PUT     /student/<int:id>/
   查询一个      GET     /student/<int:id>/
   查询所有      GET     /students/
'''

@csrf_exempt
def students(request):
    if request.method == 'GET':
        stus = Student.objects.all()
        ser = StudentSerializer(stus,many=True)
        return JsonResponse(ser.data,safe=False)

    elif request.method == 'POST':
        data = JSONParser().parse(request)
        ser = StudentSerializer(data = data)
        if ser.is_valid():
            ser.save()
            return JsonResponse(ser.data, safe=False)
        return JsonResponse(ser.errors, status=400)
@csrf_exempt
def student(request,id):
    try:
        stu = Student.objects.get(pk=id)
    except Student.DoesNotExist:
        return JsonResponse({'error': '学生不存在'}, status=404)
    if request.method == 'GET':
        ser = StudentSerializer(stu)
        return JsonResponse(ser.data)
    elif request.method == 'PUT':
        data = JSONParser().parse(request)
        ser = StudentSerializer(stu,data)
        if ser.is_valid():
            ser.save()
            return JsonResponse(ser.data, safe=False)
    elif request.method == 'DELETE':
        stu.delete()
```

```
        return HttpResponse(status=204)


```

优化

```
from django.http import JsonResponse
from rest_framework.views import APIView
from .models import Student
from .serializers import StudentSerializer
from rest_framework.response import Response


class StudentList(APIView):
    def get(self,request,format=None):
        stus = Student.objects.all()
        ser = StudentSerializer(stus,many=True)
        return Response(ser.data)

    def post(self,request,format=None):
        """
        增加直接传json
        :param request:
        :param format:
        :return:
        """
        ser = StudentSerializer(request.data)
        if ser.is_valid():
            ser.save()
            return Response(ser.data)
        else:
            return Response(ser.errors,status=400)


class StudentDetail(APIView):

    def get(self,request,pk,format=None):
        try:
            stu = Student.objects.get(pk=pk)
        except Student.DoesNotExist:
            return JsonResponse({'error': '学生不存在'}, status=404)
        ser = StudentSerializer(stu)
        return Response(ser.data,status=200)
    def put(self, request, pk, format=None):
        try:
            stu = Student.objects.get(pk=pk)
        except Student.DoesNotExist:
            return JsonResponse({'error': '学生不存在'}, status=404)
        ser = StudentSerializer(stu,data = request.data)
        if ser.is_valid():
            ser.save()
            return Response(ser.data,status=201)
    def delete(self, request, pk, format=None):
```

```
        try:
            stu = Student.objects.get(pk=pk)
        except Student.DoesNotExist:
            return JsonResponse({'error': '学生不存在'}, status=404)

        stu.delete()
        return Response({'msg': '删除成功'}, status=204)
```

GenericAPIView

继承自APIView，主要增加了操作序列化器和数据库查询的方法，作用是为下面Mixin扩展类的执行提供方法支持。通常在使用时，可搭配一个或多个Mixin扩展类

属性

- serializer_class 指明视图使用的序列化器
- queryset 指明使用的数据查询集

方法

- get_serializer_class(self) 返回序列化器类
- get_serializer(self, args, *kwargs) 返回序列化器对象
- get_queryset(self) 返回视图使用的查询集
- get_object(self) 返回视图所需的模型类数据对象

你问：**为什么 `get(self, request, pk, ...)` 中接收了 `pk`，但在 `self.get_object()` 里却没有显式传入 `pk`，却还能正确获取对象？**

因为 `GenericAPIView` **会自动从 URL 中提取** `pk` **或** `slug`，并用它去查询 `queryset`，所以你 **不需要手动传** `pk` **给** `get_object()`。

Django 会把 `pk=3` 作为关键字参数传递给视图函数或类视图的 `dispatch` 方法，最终保存在：

```
self.kwargs['pk']  # 值为 3
```

而 `get_object()` 就是从 `self.kwargs['pk']` 拿到这个值的。

`get_object()` 已经封装好了这个逻辑，它会自动从 `self.kwargs` 中提取 `pk`。

```
class StudentDetail(GenericAPIView):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    def get(self, request, pk, format=None):
        # 注意：虽然你写了 pk 参数，但 get_object() 会自动用它
        stu = self.get_object()  # ✅ 自动使用 self.kwargs['pk']
        ser = self.get_serializer(stu)
        return Response(ser.data)
```

✅ 总结

问题	回答
为什么 `get_object()` 不需要传 `pk`？	因为 `GenericAPIView` 会自动从 `self.kwargs['pk']` 中读取
`pk` 从哪里来？	从 URL 路由 `<int:pk>` 传入，保存在 `self.kwargs`
`get_object()` 做了什么？	自动用 `pk` 查询 `queryset`，找不到就返回 404
我需要手动传 `pk` 吗？	❌ 不需要，`get_object()` 已经封装好了

整体优化：

```
from django.http import JsonResponse
from rest_framework.views import APIView
from .models import Student
from .serializers import StudentSerializer
from rest_framework.response import Response
from rest_framework.generics import GenericAPIView


class StudentList(GenericAPIView):
    # 指定查询集
    queryset = Student.objects.all()
    # 指定序列化
    serializer_class = StudentSerializer

    def get(self, request, format=None):
        # 获取数据集
        stus = self.get_queryset()
        #序列化
        ser = self.get_serializer(stus, many=True)
        return Response(ser.data)

    def post(self, request, format=None):
        """
        增加直接传json
        """
        ser = self.get_serializer(data = request.data)
        if ser.is_valid():
            ser.save()
            return Response(ser.data)
        else:
            return Response(ser.errors, status=400)


class StudentDetail(GenericAPIView):
    queryset = Student.objects.all()
    serializer_class = StudentSerializer

    def get(self, request, pk, format=None):
        try:
```

```
            stu = self.get_object()
        except Student.DoesNotExist:
            return JsonResponse({'error': '学生不存在'}, status=404)
        ser = self.get_serializer(stu)
        return Response(ser.data, status=200)

    def put(self, request, pk, format=None):
        try:
            stu = self.get_object()
        except Student.DoesNotExist:
            return JsonResponse({'error': '学生不存在'}, status=404)
        ser = self.get_serializer(stu, data=request.data)
        if ser.is_valid():
            ser.save()
            return Response(ser.data, status=201)

    def delete(self, request, pk, format=None):
        try:
            stu = Student.objects.get(pk=pk)
        except Student.DoesNotExist:
            return JsonResponse({'error': '学生不存在'}, status=404)

        stu.delete()
        return Response({'msg': '删除成功'}, status=204)

```

django报错

CSRF verification failed.

`<h1>Forbidden <span>(403)</span></h1>`

`CSRF verification failed. Request aborted.`

csrf验证问题

在views中跳过验证

```
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
# Create your views here.
def students(request):
```

解析反序列化

```
data = JSONParser().parse(request)
ser = StudentSerializer(stu,data=data)
```

✅ 第一行：`data = JSONParser().parse(request)`

作用：

把 HTTP 请求体（Request Body）里的 JSON 数据，解析成 Python 的字典（dict）对象。

详细解释：

- 客户端（比如前端页面、Postman）发送的请求体是 **原始的 JSON 字符串**，例如：

  ```
  { "name": "Tom", "age": 20 }
  ```

- 但 Python 后端不能直接操作字符串，需要把它转换成 Python 的 `dict` 类型：

  ```
  {'name': 'Tom', 'age': 20}
  ```

- `JSONParser().parse(request)` 就是做这个转换的：
  - 它读取 `request` 对象中的请求体内容。
  - 解析成 Python 字典。
  - 返回这个字典，赋值给变量 `data`。

`ser = StudentSerializer(stu, data=data)`

作用：

创建一个序列化器实例，准备用新的数据（data）去更新一个已存在的模型实例（stu）。

参数	含义
`stu`	要更新的数据库对象（Student 模型实例），比如 id=3 的那个学生

参数	含义
`data=data`	客户端发来的新数据（Python 字典），表示“想改成什么样”

关键点：`data=data` 的作用

- 如果你 **不传** `data`，序列化器只是用来“读取”数据（比如做序列化返回给前端）。
- 如果你 **传了** `data`，表示你要进行 **反序列化（deserialization）** —— 把前端数据“填充”进序列化器，准备验证并保存到数据库。

`JsonResponse` 和 `HttpResponse`

是 Django 中用于返回 HTTP 响应的两个重要类。它们都属于 `django.http` 模块，但用途和行为有所不同。

`HttpResponse` 是 Django 中最基本的响应类，用于返回任意内容的 HTTP 响应。

`JsonResponse` 是 `HttpResponse` 的子类，**专门用于返回 JSON 格式的响应。**

JsonResponse(ser.data, safe=False)

参数	含义
`safe=True`（默认）	只允许 **字典（dict）** 类型的数据
`safe=False`	允许 **任何可 JSON 序列化的类型**，如列表、字符串、数字等

django报错

CSRF verification failed.

`<h1>Forbidden <span>(403)</span></h1>`

`CSRF verification failed. Request aborted.`

csrf验证问题

在views中跳过验证

```
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
# Create your views here.
def students(request):
```

解析反序列化

```
data = JSONParser().parse(request)
ser = StudentSerializer(stu,data=data)
```

✅ 第一行：`data = JSONParser().parse(request)`

作用：

把 HTTP 请求体（Request Body）里的 JSON 数据，解析成 Python 的字典（dict）对象。

详细解释：

- 客户端（比如前端页面、Postman）发送的请求体是 **原始的 JSON 字符串**，例如：

  ```
  { "name": "Tom", "age": 20 }
  ```

- 但 Python 后端不能直接操作字符串，需要把它转换成 Python 的 `dict` 类型：

  ```
  {'name': 'Tom', 'age': 20}
  ```

- `JSONParser().parse(request)` 就是做这个转换的：
  - 它读取 `request` 对象中的请求体内容。
  - 解析成 Python 字典。
  - 返回这个字典，赋值给变量 `data`。

`ser = StudentSerializer(stu, data=data)`

作用：

创建一个序列化器实例，准备用新的数据（data）去更新一个已存在的模型实例（stu）。

参数	含义
`stu`	要更新的数据库对象（Student 模型实例），比如 id=3 的那个学生

参数	含义
`data=data`	客户端发来的新数据（Python 字典），表示“想改成什么样”

关键点：`data=data` 的作用

- 如果你 **不传** `data`，序列化器只是用来“读取”数据（比如做序列化返回给前端）。
- 如果你 **传了** `data`，表示你要进行 **反序列化（deserialization）** —— 把前端数据“填充”进序列化器，准备验证并保存到数据库。

`JsonResponse` 和 `HttpResponse`

是 Django 中用于返回 HTTP 响应的两个重要类。它们都属于 `django.http` 模块，但用途和行为有所不同。

`HttpResponse` 是 Django 中最基本的响应类，用于返回任意内容的 HTTP 响应。

`JsonResponse` 是 `HttpResponse` 的子类，**专门用于返回 JSON 格式的响应。**

JsonResponse(ser.data, safe=False)

参数	含义
`safe=True`（默认）	只允许 **字典（dict）** 类型的数据
`safe=False`	允许 **任何可 JSON 序列化的类型**，如列表、字符串、数字等

django orm objects没有提示并且报黄色警告

https://blog.csdn.net/Z_Gleng/article/details/124066432

1. setting
2. Language & Framework
3. Django
4. Enable Django Support